

Roteiro prático sobre associação de objetos

Objetivo: praticar a associação entre objetos em OO

Parte II – Associação de objetos

Antes de começar, segue um exemplo de código base da classe Ponto, suficiente para iniciar e executar esse roteiro prático:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace model
{
    public class Ponto
    {
        private int x;
        private int y;

        public Ponto()
        {
            x = 0; y = 0;
        }

        public Ponto(int x, int y)
        {
            this.x = x; this.y = y;
        }
        public int getX()
        {
            return x;
        }

        public void setX(int x)
        {
            this.x = x;
        }

        public void imprimirDadosPonto()
        {
            Console.WriteLine("x:" + x + " y:" + y);
        }

        public string ToString()
        {
            return "(" + x + "," + y + ")";
        }
    }
}
```

ETAPA 1) *Refactoring* da classe Ponto para PontoVO:

- 1.1) Para executar esse roteiro prático, será necessário que você abra o projeto onde criou a classe Ponto, de RP3;
- 1.2) Seguindo as boas práticas de programação em POO e introdução aos padrões de projetos, primeiramente faça *refactoring* da classe Ponto para o nome PontoVO;
- 1.3) Teste a criação de objetos do tipo PontoVO;

ETAPA 2) Definição da classe ConjuntoVO:

2.1 – Adicione uma classe com o nome **ConjuntoVO.java**, ou **ConjuntoVO.cs** para programação em C#, ao seu projeto que deve ter os atributos código (inteiro), descricao (texto) e listaDePontoVOs, que contém uma lista de referências para objetos do tipo PontoVO. Como sugestão você pode utilizar a estrutura de dados do tipo List<PontoVO> para guardar as referências de objetos do tipo PontoVO.

2.2 – Inclua os atributos todos privados por enquanto;

2.3 – Crie os métodos construtores de forma adequada. Os valores dos atributos código e descricao deverão ser informados no momento da criação dos objetos. A lista de PontoVOs deverá ser iniciada vazia.

2.4 – Crie os métodos gets e sets para a classe;

2.5 – Crie o método toString para a classe ConjuntoVO;

2.6 – Inclua na classe ConjuntoVO os seguintes métodos:

```
public void adicionar(PontoVO PontoVO){  
    // adicione o objeto PontoVO ao objeto do tipo List e retorne o número de objetos  
    // adicionados.  
}  
  
public void excluir(PontoVO PontoVO){  
    // exclui o objeto PontoVO do List e retorne o número de objetos excluídos.  
}  
  
public void listarPontos(){  
    // Imprime as informações (toString) de todos os objetos PontoVO na tela  
}
```

2.7 – No Main do seu projeto crie objetos do tipo PontoVO e os associe com outros objetos do tipo ConjuntoVO;

2.8 – Teste os métodos adicionar, listar e excluir objetos do tipo PontoVO de um objeto do tipo ConjuntoVO;

ETAPA 3) Definição da classe ConjuntoHelper:

3.1 – Acrescente ao seu projeto a classe ConjuntoHelper que contém os atributos código (inteiro), descricao (string) e uma lista de conjuntosVOs (objetos do tipo collection);

3.2 – Faça as adaptações para que a classe ConjuntoHelper implemente um singleton;

3.3 – Crie uma interface amigável para o usuário que permita ao usuário criar vários objetos do tipo ConjuntoVO e guarda-los em memória;

3.4 – Crie uma interface amigável para o usuário que permita ao usuário criar vários objetos do tipo PontoVO e associá-los a objetos do tipo ConjuntoVO previamente cadastrados;

3.5 – Faça uso de todas as boas práticas estudadas até aqui: tratamento de exceções, modularização, etc.

Bom trabalho.